

## 1 Method of Moments, Maximum Likelihood, and Moment Generating Functions

In this assignment we explore three classical estimation techniques on a simple parametric model, and then investigate why higher-order moment matching may fail in practice. The exercises combine theoretical derivations and empirical analysis.

### The shifted exponential distribution

Consider the shifted exponential distribution with parameters  $a \in \mathbb{R}$  and  $\lambda > 0$ , with density

$$f_{a,\lambda}(x) = \lambda e^{-\lambda(x-a)} \mathbf{1}_{\{x \geq a\}}.$$

Let  $X \sim f_{a,\lambda}$ . Throughout the section assume we observe an i.i.d. sample  $X_1, \dots, X_n$  from this distribution.

### Method of Moments

1. Derive the first two theoretical moments of  $X$ , i.e.  $\mathbb{E}[X]$  and  $\mathbb{E}[(X - \mathbb{E}[X])^2]$ , and express them as functions of  $(a, \lambda)$ .

2. Let

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i, \quad S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X}_n)^2$$

be the empirical mean and (non-unbiased) variance. Set the theoretical moments equal to the empirical ones and solve this system to obtain the moment estimators  $\hat{a}_{\text{MoM}}$  and  $\hat{\lambda}_{\text{MoM}}$ .

### Maximum Likelihood Estimation

3. Write the log-likelihood  $\ell(a, \lambda)$  of the sample.
4. Show that the MLE of  $a$  is  $\hat{a}_{\text{MLE}} = \min_i X_i$ .
5. Derive the corresponding MLE  $\hat{\lambda}_{\text{MLE}}$  by maximizing the log-likelihood w.r.t.  $\lambda$ .

### Simulation study

Let  $a^* = 2$  and  $\lambda^* = 0.7$ .

6. Generate a sample of size  $n = 100$  from the shifted exponential with parameters  $(a^*, \lambda^*) = (0, 1)$ , compute the two MoM estimators and the two MLE estimators, and report the values.

7. Repeat the experiment 5000 times and produce two histograms:

- the empirical distribution of the estimator of  $a$ ,
- the empirical distribution of the estimator of  $\lambda$ ,

comparing Method of Moments and MLE. Print out the mean and variance of the two MoM estimators and the two MLE estimators over 5000 experiments.

8. Interpret the results: which estimators appear more accurate? Which have lower variance? Give a short theoretical explanation for what you observe.

### MGFs : the Gaussian example

In Part 1 you used raw moments via empirical formulas; here you see how MGFs let you get the moments of an important distribution in closed form. For instance, moments of the Gaussian distribution are not easy to obtain from the integral definition, because Gaussian integrals do not have elementary antiderivatives and require several algebraic manipulations.

To illustrate the usefulness of MGFs as a theoretical tool, consider

$$X \sim \mathcal{N}(\mu, \sigma^2), \quad M_X(t) = \exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right).$$

9. Using this MGF, compute  $\mathbb{E}[X]$  and  $\mathbb{E}[X^2]$  by differentiating  $M_X(t)$  at  $t = 0$ , and verify that the variance of  $X$  is  $\sigma^2$ . (You may attempt the integrals directly to see why the MGF method is much simpler, but this is not required.)

10. Explain when MGFs can be useful.

## 2 Chain rules for probabilities

For each of the 3 following chains of information, give the joint probability using the chain rule:

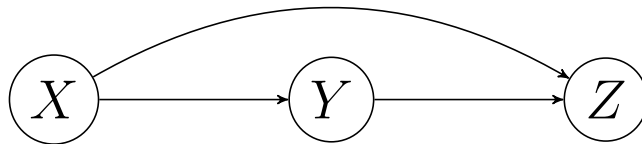
1.

$$p(X, Y, Z) = \dots?$$

$$p(X, Z) = \dots?$$

$$p(X, Y) = \dots?$$

$$p(Y, Z) = \dots?$$



2.

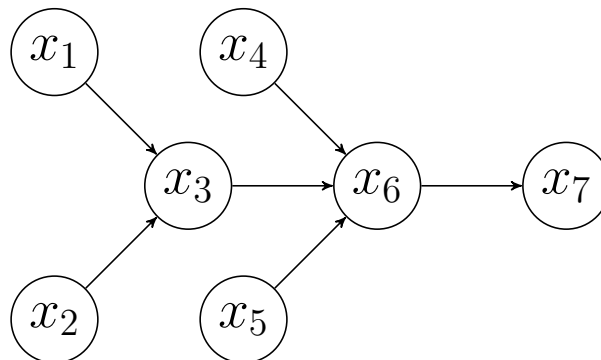
$$p(X_1, X_2, \dots, X_7) = \dots?$$

$$p(X_1, X_3, X_5, X_7) = \dots?$$

$$p(X_2, X_4, X_6, X_7) = \dots?$$

$$p(X_3, X_6, X_7) = \dots?$$

$$p(X_1, X_2, X_4, X_5) = \dots?$$



3.

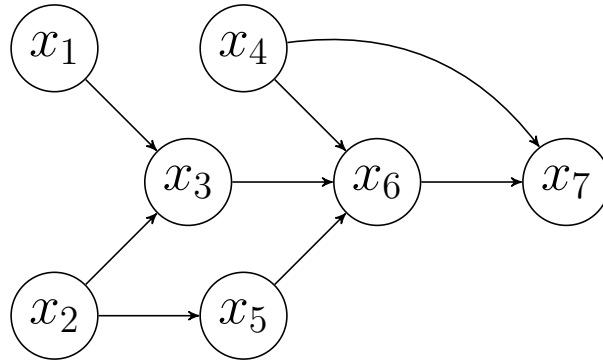
$$p(X_1, X_2, \dots, X_7) = \dots?$$

$$p(X_1, X_3, X_5, X_7) = \dots?$$

$$p(X_2, X_4, X_6, X_7) = \dots?$$

$$p(X_3, X_6, X_7) = \dots?$$

$$p(X_1, X_2, X_4, X_5) = \dots?$$



### 3 Problem: information quantifiers

U, V and W are three binary random variables. Their joint probability mass function is depicted as below:

| U | V | W | $p_{U,V,W}(u, v, w)$ |
|---|---|---|----------------------|
| 0 | 0 | 0 | $\frac{1}{4}$        |
| 0 | 0 | 1 | 0                    |
| 0 | 1 | 0 | $\frac{1}{4}$        |
| 0 | 1 | 1 | $\frac{1}{8}$        |
| 1 | 0 | 0 | 0                    |
| 1 | 0 | 1 | $\frac{1}{8}$        |
| 1 | 1 | 0 | 0                    |
| 1 | 1 | 1 | $\frac{1}{4}$        |

Calculate the information measures below:

1.  $H(U)$ ,  $H(V)$  and  $H(W)$
2.  $H(U|V)$ ,  $H(V|U)$  and  $H(W|U)$
3.  $I(U; V)$ ,  $I(U; W)$  and  $I(V; W)$

#### 4. $H(U, V, W)$

For each of the above items you could directly use the definition. However, because they are related to each other in many ways, you could calculate some of them and derive the rest by using their relations: chain rules for entropy and mutual information, the Venn diagrams.

## 4 Problem: Source Coding

We study a binary memoryless source  $X$ , where

$$P(X = 1) = \theta = 0.1, \quad P(X = 0) = 0.9.$$

### Tasks

#### 1. Data Generation:

- Generate  $n = 10,000$  binary samples.
- Group them into symbols of length 5 (2000 total symbols).
- Count the frequency of each symbol.

#### 2. Huffman Coding:

- Build a Huffman code (e.g. with `dahuffman`) using the symbol frequencies.
- Encode the sequence and record its total length in bits.

#### 3. Entropy and Efficiency:

- Compute the theoretical entropy  $H(X)$ .
- Compute the average Huffman code length:

$$L = \frac{\text{encoded length (bits)}}{\text{number of symbols}}.$$

- Compare  $H(X)$  and  $L$ . Comment on compression efficiency.

## 5 Problem: Communication System with Noise

We send a binary sequence through a channel with additive white Gaussian noise (AWGN).

### System Setup

- A '1' is transmitted as amplitude  $A = 1$ , a '0' as  $A = 0$ .
- Each bit is repeated  $N$  times ( $N = 10$  at first).
- The channel adds Gaussian noise  $z[n] \sim \mathcal{N}(0, \sigma_z^2)$  with  $\sigma_z^2 = 1.5$ .

### Tasks

1. Signal Generation: Generate  $k = 10,000$  random bits, build the transmitted signal  $x[n]$ , and add AWGN to get  $y[n]$ .
2. Detection Rules:
  - Derive the Neyman–Pearson rule with false alarm  $P_{FA} = 0.01$ .
  - Derive the Bayesian rule assuming  $P(H_0) = P(H_1) = 0.5$ .
  - Report both thresholds  $\gamma_{NP}$  and  $\gamma_{Bayes}$ . Compare them.
3. Apply Detection: Use both rules to estimate the received bits. Compare with the original sequence.
4. Visualization: Plot  $x[n]$ ,  $y[n]$ , and the estimated  $\hat{x}[n]$ . Zoom in on 100 samples to see the noise effect.
5. Performance: Compute empirical  $P_M$  (miss),  $P_{FA}$  (false alarm), and  $P_e$  (total error). Compare with theoretical values.
6. Effect of  $N$ : Vary  $N$  from 1 to 100. Plot how  $P_M$ ,  $P_{FA}$ , and  $P_e$  change for both detection rules. Explain why larger  $N$  improves detection.

## 6 (Small project (*optional*)) When moment matching fails

In this section we study a simple nonlinear simulator and investigate how different moment-matching losses behave in practice.

Consider the generator

$$X = g_\theta(Z) = \theta_0 + \theta_1 Z^2 + \theta_2 Z^5, \quad Z \sim \mathcal{N}(0, 1),$$

with true parameters

$$\theta^* = (2, 0.7, 0.05).$$

### Data generation

Generate  $n = 10\,000$  samples  $X_1, \dots, X_n$  from the model using the true parameters. Keep these samples fixed for all subsequent experiments (i.e. do not regenerate new data each iteration).

### (b) Optimization

Let  $\theta = (\theta_0, \theta_1, \theta_2)$  be a trainable parameter vector in PyTorch, initialized at

$$\theta^{(0)} = (1.6, 0.7, 0.01).$$

Use the Adam optimizer with learning rate  $10^{-3}$ . At each iteration, draw a fresh batch  $Z \sim \mathcal{N}(0, 1)$  of size 1024 and compute  $X_{\text{model}} = g_\theta(Z)$ .

Unless stated otherwise, run all experiments below for 10 000 iterations.

Train the parameter vector  $\theta$  using the following losses, one at a time:

#### 1. Match only the mean:

$$L_1 = (\mathbb{E}[X_{\text{model}}] - \mathbb{E}[X_{\text{data}}])^2.$$

#### 2. Match mean and variance:

$$L_2 = (\mathbb{E}[X_{\text{model}}] - \mathbb{E}[X_{\text{data}}])^2 + (\text{Var}(X_{\text{model}}) - \text{Var}(X_{\text{data}}))^2.$$

#### 3. Match mean, variance, and 5th central moment:

$$L_3 = L_2 + (\mathbb{E}[(X_{\text{model}} - \mathbb{E}[X_{\text{model}}])^5] - \mathbb{E}[(X_{\text{data}} - \mathbb{E}[X_{\text{data}}])^5])^2.$$

After training with each loss, record the final value of  $\theta$ .

## Kernelized moment matching (MMD)

Define the Gaussian kernel

$$k(x, y) = \exp\left(-\frac{(x - y)^2}{2\sigma^2}\right), \quad \sigma = 2.$$

Define the empirical squared MMD between two batches  $x$  and  $y$  as

$$\text{MMD}^2(x, y) = \frac{1}{m^2} \sum_{i,j} k(x_i, x_j) + \frac{1}{m^2} \sum_{i,j} k(y_i, y_j) - \frac{2}{m^2} \sum_{i,j} k(x_i, y_j).$$

---

```
1 def rbf_kernel(x, y, sigma=1.0):
2     x = x.view(-1, 1)
3     y = y.view(1, -1)
4     diff2 = (x - y)**2
5     return torch.exp(-diff2 / (2 * sigma**2))
6
7 def mmd_squared(x, y, sigma=1.0):
8     Kxx = rbf_kernel(x, x, sigma)
9     Kyy = rbf_kernel(y, y, sigma)
10    Kxy = rbf_kernel(x, y, sigma)
11    m = x.shape[0]
12    n = y.shape[0]
13    return Kxx.mean() + Kyy.mean() - 2 * Kxy.mean()
```

---

Now train  $\theta$  by minimizing

$$L_{\text{MMD}} = \text{MMD}^2(X_{\text{data}}, X_{\text{model}}).$$

Record the final value of  $\theta$ .

## Empirical comparison

For each loss  $L_1$ ,  $L_2$ ,  $L_3$ , and  $L_{\text{MMD}}$ :

1. Generate 10 000 new samples from  $g_\theta(Z)$  using the trained parameter vector.
2. Plot a histogram of this model-generated sample along with a histogram of the real data sample.
3. Compare the fitted parameter vector  $\theta$  with the true value  $\theta^*$ .



---

```
1 _, bins, _ = plt.hist(
2     x_data.numpy(), bins=50, density=True,
3     alpha=0.5, label="data"
4 )
5 plt.hist(
6     x_model_final.numpy(), bins=bins, density=True,
7     alpha=0.5, label="model (MMD trained)"
8 )
```

---

## Interpretation

Provide a concise discussion:

- Which losses successfully recover  $(\theta_0, \theta_1)$ ?
- What happens when including the 5th central moment? Why does this lead to unstable training or incorrect  $\theta_2$ ?
- Why does the MMD loss (implicitly matching infinitely many *smooth* bounded moments) produce stable and accurate estimates?
- How does this illustrate the general difference between matching raw high-order moments and using smooth kernel-based moments?

## Submission

Please archive your report and code (*do **not** include dataset*) in “Prénom Nom.zip” (replace “Prénom” and “Nom” with your real name), and upload to “TP-D: Probabilities, Information Theory, and Statistical Estimation.” on <https://moodle.unige.ch> before **Sunday, December 7 2025, 21:59 PM**. Note, that the assessment is mainly based on your report, which should include your answers to all questions and the experimental results. *Importance is given on the mathematical explanations of your works and your codes should be commented*

**Please use this [overleaf template](#).**